HACPO: Optimized Specification

This file refines the original HACPO (Hybrid Adaptive Contrastive Policy Optimization) objective: we 1) unify notation, 2) make clipping/PPO mechanics explicit, 3) define the contrastive term and weights, 4) give practical hyperparameter suggestions and 5) provide implementation notes and pseudocode.

Notation (unified).

- θ: policy parameters for $\pi_\theta(y \mid x)$.
- π_{ref} : reference / behavior policy (fixed when computing ratios).
- $r_i(\theta) = \dfrac{\pi_\theta(y_i \mid x_i)}{\pi_{\text{ref}}(y_i \mid x_i)}$: importance / policy ratio for supervised sample (x_i, y_i).
- $r_j^{(k)}(\theta) = \dfrac{\pi_\theta(y_{j,k} \mid x_j)}{\pi_{\text{ref}}(y_{j,k} \mid x_j)}$: ratio for the k-th contrastive (negative) sample associated with context x_j.
- $\hat{A}_i$: estimated (possibly generalized) advantage for sample i. Use the same notation for all advantages for clarity.
- $w_{j,k} \geq 0$: weight for contrastive sample $y_{j,k}$; we require $\sum_k w_{j,k} = 1$ for each context j.
- Clip interval parameter $\epsilon > 0$; define $\text{clip}_\epsilon(r) = \text{clip}(r, 1-\epsilon, 1+\epsilon)$.
- L_{CPO} : contrastive policy optimization loss (defined below).
- $D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$: KL divergence used as a constraint/penalty. (See note about direction and sign.)

Objective

We present the HACPO objective in a form consistent with PPO-style clipping and with an explicit contrastive term.

$$
\begin{aligned}
J_{\text{HACPO}}(\theta) = \mathbb{E}_{(x_i,y_i)\sim D}\Bigg[& \alpha_i \cdot \underbrace{\min\left(r_i(\theta)\hat{A}_i,\ \text{clip}_\epsilon(r_i(\theta))\hat{A}_i\right)}_{\text{supervised/PPO-clip term}} \\
& + (1-\alpha_i)\cdot \underbrace{\sum_{k=1}^{K} w_{i,k} \min\left(r_i^{(k)}(\theta)\hat{A}_i^{(k)},\ \text{clip}_\epsilon(r_i^{(k)}(\theta))\hat{A}_i^{(k)}\right)}_{\text{contrastive importance-sampled PPO-clip}} \Bigg] \qquad (1) \\
& + \gamma\, \mathbb{E}_{x_i\sim D,\{y_{i,k}\}\sim A}\left[L_{\text{CPO}}(\theta; x_i, \{y_{i,k}\}) \right] \\
& - \beta_{\text{KL}}\, D_{\text{KL}}(\pi_{\text{ref}} \,\|\, \pi_\theta).
\end{aligned}
$$

Notes on signs and KL direction: Many RL formulations either maximize an objective (PPO) or minimize a loss. Above we write J_{HACPO} as an objective to maximize; the KL term is written as a penalty by subtracting $\beta_{\text{KL}} D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$. If you implement using gradient descent on a loss, negate J_{HACPO} accordingly. We use $D_{\text{KL}}(\pi_{\text{ref}} \| \pi_\theta)$ so that the penalty encourages π_θ to stay close to the reference policy [?].

Contrastive loss: L_{CPO}

One practical choice for the contrastive policy loss (inspired by InfoNCE-style contrastive objectives, adapted to policies) is:

$$
L_{\text{CPO}}(\theta; x, \{y_k\}) = -\log \frac{\exp\left(s_\theta(x, y^+)/\tau_c\right)}{\exp\left(s_\theta(x, y^+)/\tau_c\right) + \sum_{k=1}^{K} \exp\left(s_\theta(x, y_k^-)/\tau_c\right)}, \qquad (2)
$$

where:

- y^+ is the positive (target) sample (usually the reference y),

- y_k^- are negative samples drawn from A (or from the current mini-batch),
- $s_\theta(x,y) = \log \pi_\theta(y \mid x)$ (or any scalar scoring function correlated with log-probability),
- $\tau_c > 0$ is a temperature for the contrastive softmax.

This choice encourages the policy to assign higher relative probability to positive examples than to negatives. Alternatively, you can use margin-based or pairwise losses depending on your task.

Weights $w_{i,k}$ and negative sampling strategy

- Uniform weights: $w_{i,k} = 1/K$ is simple and stable.
- Similarity-based weights: compute similarity scores (e.g. using s_θ or an auxiliary encoder) and convert to weights via softmax:

$$w_{i,k} = \frac{\exp\left(\varphi(x_i, y_{i,k})\right)}{\sum_{\ell=1}^{K} \exp\left(\varphi(x_i, y_{i,\ell})\right)}.$$

  This emphasizes "hard" negatives.
- Batch negative strategy: use other examples in the mini-batch as negatives to avoid extra sampling overhead.

Always normalize $w_{i,k}$ so $\sum_k w_{i,k} = 1$ for numerical stability.

Adaptive mixture coefficient α_i

To adapt the relative importance of the supervised vs contrastive terms, define α_i per sample. One robust choice:

$$\alpha_i = \sigma\left(\frac{\ell(x_i, y_i) - \tau}{\beta'}\right), \tag{3}$$

where $\ell(x,y)$ is a sample-level signal (e.g. token length $|y|$, or a difficulty score such as negative log-likelihood under reference policy), τ a threshold, and β' a temperature controlling smoothness. Practical recommendations:

- If $\ell = |y|$ (sequence length), tune τ on a validation set (512 is task dependent).
- Consider alternate signals: model confidence, reference log-prob, or validation loss per example.
- Optionally clip $\alpha_i \in [\alpha_{\min}, \alpha_{\max}]$ to avoid extremes.
- You can also schedule α over training steps (e.g. slowly move from supervised-heavy to contrastive-heavy).

Hyperparameter defaults & practical guidance

- ϵ (PPO clip): common values 0.1-0.2; start with 0.1.
- K (negatives per context): 4-16; prefer batch negatives when K large.
- γ (weight for L_{CPO}): start with 0.1-0.5; tune to balance supervised vs contrastive objectives.
- β_{KL}: if used, adaptively adjust as in PPO-penalty variants: if observed KL > target, increase β_{KL}; otherwise decrease.
- Optimizer: Adam / AdamW with learning rate tuned separately for policy and for any encoder used by contrastive loss.
- Advantage estimation: use GAE (generalized advantage estimation) to reduce variance.

Stability considerations and implementation tricks

- **Gradient conflicts:** supervised and contrastive gradient directions can conflict. Consider alternating updates (one step supervised PPO-clip, one step contrastive) or use gradient surgery techniques if conflicts are severe.
- **Clipping and zero gradients:** when ratios are clipped, gradient contributions from those samples are limited. Monitor fraction of clipped updates; if too high, reduce learning rate or increase ϵ.
- **Value normalization:** normalize advantages (zero mean, unit variance) per batch to stabilize scale.
- **Use mini-batch vectorization:** compute ratios and clip operations in a vectorized way for speed.
- **Monitoring:** track (1) average KL, (2) fraction clipped, (3) contrastive loss magnitude, (4) supervised PPO loss magnitude. Keep their scales comparable by tuning γ and β_{KL}.

Pseudocode (sketch)

Algorithm 1 HACPO Training Step (sketch)

Require: dataset $\mathcal{D}$, contrastive generator $\mathcal{A}$, batch size B, negatives K

for each training iteration **do**
  Sample batch $\{(x_i, y_i)\}_{i=1}^{B}$ from $\mathcal{D}$
  For each i, sample negatives $\{y_{i,k}\}_{k=1}^{K} \sim \mathcal{A}$ (or use batch negatives)
  Compute reference probabilities / log-probs under π_{ref}
  Compute current log-probs under π_θ and ratios r_i, $r_i^{(k)}$
  Compute advantages $\hat{A}_i$, and (optionally) $\hat{A}_i^{(k)}$ for negatives
  Compute α_i per Eq.(3)
  Compute supervised PPO-clip term and contrastive PPO-clip term (vectorized)
  Compute L_{CPO} per Eq.(2)
  $J \leftarrow$ average objective per Eq.(1)
  Take gradient step to maximize J (or minimize $-J$)
  Update β_{KL} adaptively if using KL-penalty scheme
end for

Summary of improvements over original spec

- Unified notation for ratios and advantages to avoid ambiguity.
- Explicit clip interval and PPO-style clipping semantics.
- Concrete contrastive loss (InfoNCE-style) option and weight normalization.
- Practical hyperparameter defaults and stability/implementation guidance.
- Pseudocode to reduce misinterpretation at implementation time.